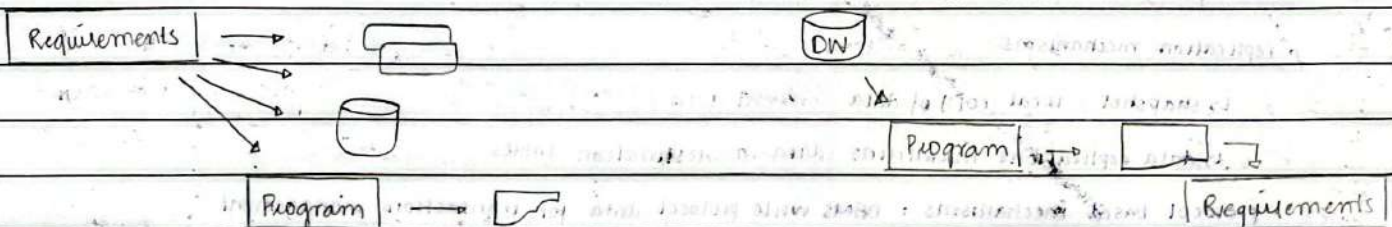


Dated:

LIFE CYCLE OF DATA WAREHOUSE

- datawarehouse SDLC is almost the opposite of classical SDLC.
- SDLC → system development life cycle.



★ classical SDLC

- requirements gathering
- analysis
- design
- programming
- testing
- integration
- implementation

★ datawarehouse SDLC / CLDS

- implement warehouse
- integrate data
- test for bias
- program against data
- design DSS system
- analyze result
- understand requirements

- SDLC is also called "waterfall" approach, because one activity upon its completion spills down into the next activity and triggers its start.
- CLDS is a data driven development life cycle. It starts with a data. Once data is at hand, it is integrated and tested against bias. Programs are written against the data. The results of the program are analyzed, and requirements of the system are understood. Once requirements are understood, adjustments are made to the design of the system. The cycle starts all over again for a different set of data. Also called "spiral" approach.
- Design → Prototype → Deploy → Operate → Enhance.

Dated:

OPERATING A DATAWAREHOUSE

1. Monitoring

- surveillance of the data sources
- identifying what, where, what rate the growth is occurring.
- replication mechanisms

— logs data.

* protocol based mechanism

* application managed mechanism

* active mechanisms (ECA)

↳ snapshot : local copy of data / present data / changes cannot be reflected.

— Sales exceed certain threshold

↳ data replication: maintains data in destination tables

- protocol based mechanisms : DBMS write protocol data for transaction management.
- application managed mechanisms : hard to implement for legacy systems.

2. Extraction

- reads the data which was selected throughout the monitoring phase.
- due to large data volume, compression can be used.
- time-point for performing extraction can be :

↳ periodical

↳ on request

↳ event driven : whatsapp gets updated at 2.00

↳ immediate :

- largely depends on hardware and the software used for the datawarehouse

3. Transforming

- implies adapting data
- data integration : handling of key attributes, conversion of encoding, adaptation of different types of same data, correcting wrong information, normalization, data handling, measurement units, aggregation
- data cleaning : consistency check, completeness. (NULL values)

4. Loading

- usually takes place during weekends or night, system not under user stress.
- split between initial load* and periodical load.
- partitioning, parallelization, incremental actualization → an old record cannot be loaded again.

* implies big volume of data

Dated:

5. Analyzing.

- organized on a multidimensional data model.
- operations include: pivoting/rotation, roll up, drill down, drill across
- data mining
- KDD, Prediction
- useful for answering questions.
- methods for data mining → clustering, classification, regression

drill down: adding a new dimension, less detailed to highly detailed.

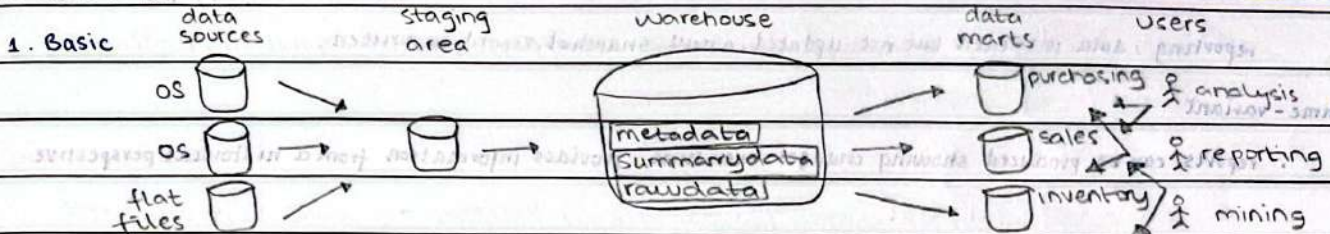
drill up: reducing dimensions (city → country)

slice: selects a single dimension, results in a new subcube.

dice: selects two or more dimensions

Pivot: rotates a current view to get a new view

ARCHITECTURE



- OS: refer to a system that is used to process day to day transaction of an organization.
- flat files: transactional data is stored, every file must have a different name.
- metadata: set of data that defines and gives information about other data.
- Summary data: speed up query performance, updated continuously.
- staging area: a place where data is processed and stored, record from source system are copied, should be accessible only to skilled professionals.
- data marts: data is organized, stored and made available for queries and other analytical processing.

DATA WAREHOUSE

- a large database, spans over several servers and needs computing power.
- a collective data repository
- contains snapshot (history) / obtained through data cleaning (ETL), for analytics.
- tactical / strategic focus.

Layered architecture → 2 tiers, analysis performed on clients end.

↳ thin client, fat client

Dated:

CHARACTERISTICS

1. subject oriented.

- data elements relating to the same realworld event or object are linked together.

2. integrated

- data from all organizations OS, and is consistent, connection between large amount of data from multiple database

3. non-volatile

- data is never over-written or deleted, once committed data is static, read only and retained for future reporting, data is loaded but not updated, a new snapshot record is written.

4. time-variant

- reports can be produced showing changes over time, provides information from a historical perspective.

OLTP

- operational data - represents day to day.

- for data entry and retrieval transaction processing.

- reflects only current state → doesn't need historical data.

OLAP

- represents front end analytics, to analyze aggregated data.

- analyzing trends
- decision oriented, takes time.

OLTP vs DW

- separate RDBMS installation that contains copies of data

- optimistic locking DBMS

- different pattern of hardware utilization between on-line and analytical processing.

→ move utilization in DB

→ offices, counter

- OLTP: mostly updates, small transactions, MB-TB data, raw data, clerical users, up to date

- DW: mostly reads data, complex long queries, GB-PB data, summarized, decision makers, outdated slightly (history)

Dated:

APPLICATION OF DATAWAREHOUSE

- SQL queries can be difficult due to complex joins and multiple scans.
- in DW, tables are rearranged and pre-aggregated → arrangement is subject-oriented.
- OLAP is a form of information processing, needs to provide timely, accurate, understandable information.

↳ OLTP : update in matter of seconds.

↳ OLAP : takes minute, hours or longer → ^{desktop}ROLAP, ^{multi}ODOLAP, ^{multi}NOLAP etc. H-hybrid.

- KDD : constructs models of the data → viewed as high level summaries.
- users are called DSS analysts → job is to define and discover info used in corporate decision making.
- DSS analysts, starts with existing requirements, but adds new requirement with time.

logical data mart :

ETL near real-time → updated simultaneously when data is received

disadvantage → server can be overloaded as all of data is stored in one place.

run-time cost is more.

generic three-tier architecture :

reconciled data, derived data (certain task that has been selected)

↳ after transformation, to ensure consistency.

conceptual model → schema, follow steps for end products.

centralized DW → data in one place

geographical distribution : locally and globally.

data modelling : → creating data model by analyzing the requirements

ted:

RE

2. Storage

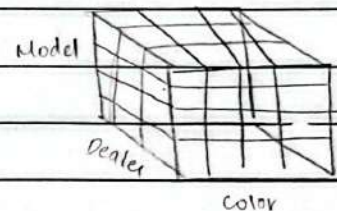
- after extraction from operational data, information is stored in databases.
- databases operated by DBMS.
- RDB operated by a RDBMS
- MultiDimensional Model operated by a MDBMS
- in staging area: RDBMS can be used but off limit to user because of performance
- default: normalized database not included in presentation area → MDBMS.

- Relational DB

- ↳ collection of predicates over a finite set of variables.
- ↳ relation is defined as a set of tuples that have same attribute.

- Multidimensional DB

- ↳ optimized for DW and OLAP application.
- ↳ uses input from staging area.
- ↳ designed for efficient and convenient storage
- ↳ retrieval of large volumes of data.
- ↳ viewed and analyzed from different perspective called dimensions.
- ↳ complexity grows quickly with the number of dimensions & positions.
- ↳ MDBs are faster than RDBMS.
- ↳ ease of data presentation, maintenance, performance.
- ↳ data is stored the same way it is viewed by attribute, process data and answer queries faster.
- ↳ examples: budgeting, promotion tracking.



Dated: _____

3. Tier

Thin client

↳ analytics are executed on the server

↳ client just displays

↳ relies on server for processing and logic

• Fat client

↳ server delivers data

↳ analytic executed on client

↳ communication between client and server able to sustain large data transfers

1. Generic Two-Tier Architecture

• data is not completely current in the datawarehouse

• periodic extraction

• staging area is present between the sources and the storage to ensure all data loaded into the

warehouse is cleaned and in appropriate format

2. Independent Data Mart

• created directly from external sources instead of datawarehouse

• datawarehouse created from data present in data mart

• separate ETL for each data mart

• used by small organizations

3. Dependent Data Mart and Operational Data Store

• created by extracting the data from central repository, datawarehouse

data marts

source → ETL → datawarehouse → analytics

• single ETL for the datawarehouse

• simple data access

• limit users access to only the data mart or allow them to access both, maintain data integrity

4. Logical Data Mart and Active Warehouse

• ETL is near real-time → update simultaneously when data received

• data marts not separate databases, but logical views of the DW

sources → staging area and storage area → end-user

processing, datawarehouse

transformation layer

data mart

Dated:

5. Generic Three Tier Architecture

- derived data - data that had been selected, formatted and aggregated
- reconciled data - detailed, current data to be the single source.

* HOSPITAL MANAGEMENT SYSTEM

1. Medicine Popularity for a specific treatment → No. of patients,
2. Capacity.
3. Doctor Performance.
4. Medical Condition.
5. Medicine Effectiveness.
6. Medical Condition based on region.
7. Revenue generated against medicine.
8. Depts / - distribution of resources.
9. Patient percentage based on gender/age →
10. Analysis on Doctor Performance.

60% mcqs.

1 long question - 30% tables. MER model.

Dated:

DISTRIBUTED DATAWAREHOUSE

- economics and technology greatly favor a single centralized datawarehouse.
- centralized → where all the data is put together
- distributed DW is useful by making replications of the data because if one is damaged, it can still have data access from other
- but because of data replication, inconsistencies can happen

TYPES OF DISTRIBUTED DW

1. Geographically

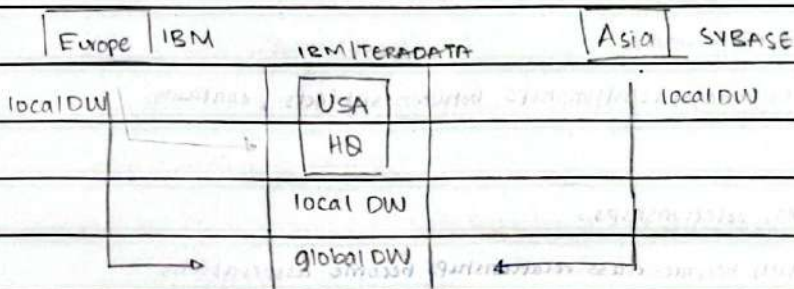
↳ local/global : distributed DW when processing occurs at local level e.g. VS should have their own DW, and UK should also.

2. Technologically

↳ logically one, but several servers that serve different parts of datawarehouse e.g. data marts.

3. Independently evolving

↳ entry cost is cheap



• as the DW starts to expand network data, communication plays an important role.

DATA MODELLING

- process of creating data model by analyzing requirements needed to support the business process.
- called database as well as the model is eventually implemented in a database.
- data models
 - ↳ definition and format of data.
 - ↳ specific within area of interest.
 - ↳ enterprise and subject area.

Dated:

PHASES

1. Conceptual Design.

↳ is high level, does not contain any implementation details.

↳ describes data entities, relationships, constraints.

↳ independent of software and hardware.

2. Logical Design

↳ maps conceptual data model to the logical data model used by DBMS.

↳ how data is stored

3. Physical Design

↳ creates internal structure needed to efficiently store data.

↳ tables, indexes, path etc.



Conceptual Model: * highest level, grouping of ideas, major relationships between subjects, contains

less details.

* contains entities, attributes, relationships

* UML: class diagrams → entity becomes class, relationships become associations

Logical Model: * arranges data into a logical structure, mapped into the storage objects supported by DBMS.

Physical Model: * data elements, data types

DATA MODEL IN DW

• managing complex data relationships

• transformation and integration from various system of record.

• provides means of supplying users with a roadmap through the data and relationships

• modelling business queries

• goal: define the purpose and decide on the subjects, identify questions of interest.

Dated:

CONCEPTUAL MODEL

- for conceptual design in DW, ER or UML are not appropriate.
- ER is for removing redundancy and facilitate retrieval of individual records.
- therefore DW optimize OLTP where insertion, updates take place.
- DW redundancy and materialized views help speed up analytical queries.
- components

↳ facts: basic entities for decision making

↳ measures: attributes that describe facts from different point of view.

↳ dimensions: discrete attributes which determine granularity to represent facts.

↳ hierarchies: made up of dimension attributes. e.g. day - month - quarter - year.

- use Multidimensional ER, UML

MULTIDIMENSIONAL ER MODEL

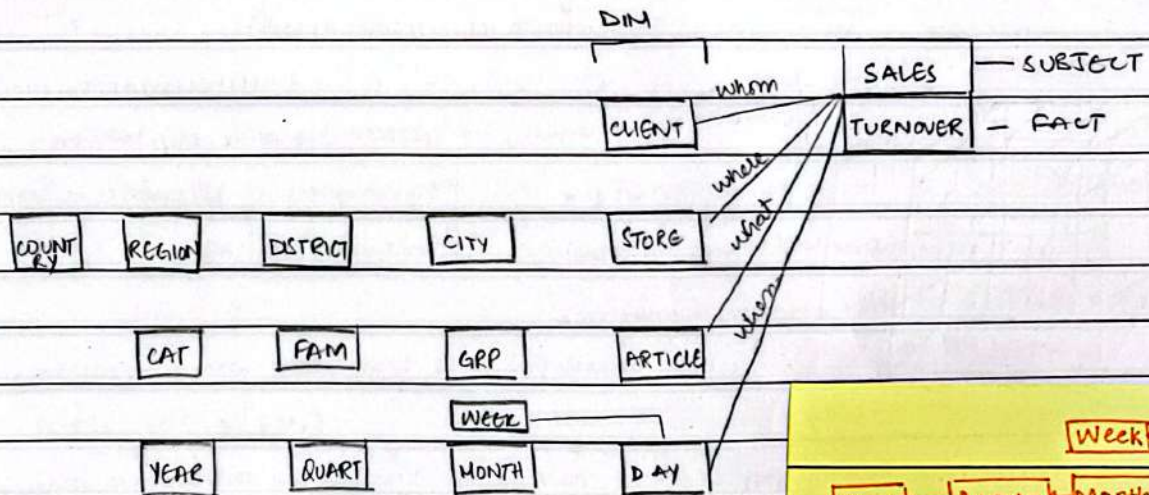
- intuitive representation of the data so that we can pick distinct aggregation step for good information quality and faster.
- represents specialization and evolution of the ER - entities, relationships.
- specialization of ER model
 - ↳ all new elements specialization of ER elements.
- minimal expansion of ER model
 - ↳ adding elements for multidimensional, but should be less.
- 3 main ME/R constructs
 - ↳ fact node: entity you're interested in, described by attributes
 - ↳ level node,
 - ↳ classification edge: tells classification level

Dated:

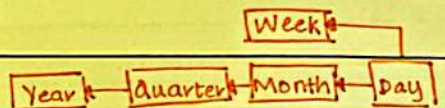
LOGICAL MODEL

→ goal.

- confirm the subject areas e.g. sales, customers, inventory.
- create facts and dimensions for each subject e.g. product dimension, time dimension.
- granularity for the dimensions.

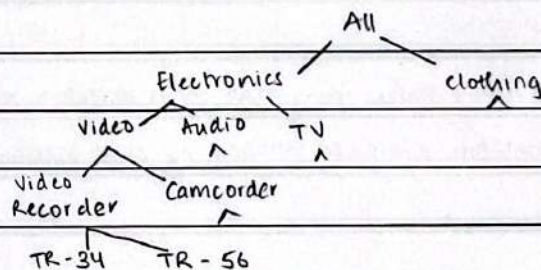


- dimensions are analysis purpose chosen entities within the data
- one dimension can be used to define more than one cube.
- dimension hierarchies are organized in classification levels.
- functional dependencies → an attribute B is functionally dependent on A if for every domain $dom(A)$ there exists exactly one b.



• 1st May, 1999 have the exact month as May cannot be in some other month, same with Year i.e. 1999

- with smallest element i.e. there is no classification level with sm
- a fully ordered set of classification level is called a path.
- a classification hierarchy is a balanced tree where the nodes are domain of all different levels and edges in the path are functional dependencies.



category
Prod Family
Prod Group
Article

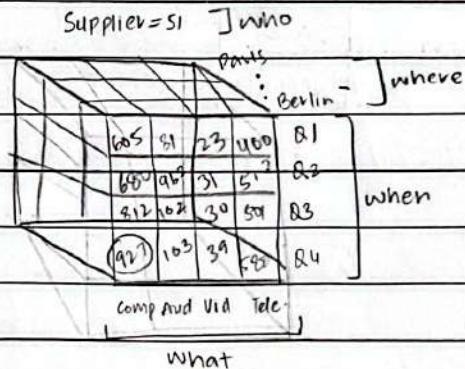
Dated:

CUBE

- consist of data cells with one or more measures.
- cube schema consists of granularity (set of dimensions) and set of measures.
- co-ordinates of cell are classification nodes from $\text{dom}(G)$ corresponding to the cell.

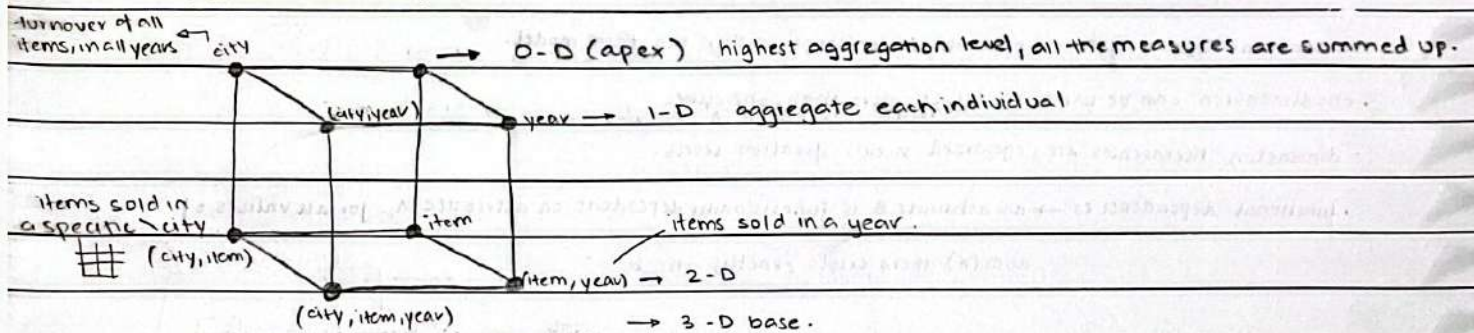
e.g. Sales ((Article, Day, Store, client), (Turnover))

↓
measure (how much did we sell)



*query → Specific supplier i.e. S1, sells over 900 computers in 1st quarter in Berlin.

- n-D is called a base cuboid. important



BASIC OPERATIONS

- selection
- projection
- cube join
- sum
- aggregation

Dated:

1. MULTIDIMENSIONAL SELECTION

σ P.Prod-group = "Video" (Sales)

↳ measures of sale → Price, sold items

↳ defines aggregation level in the dimension Products

2. MULTIDIMENSIONAL PROJECTION

π turnover, sold-items (Sales)

SALES
turnover
sold-items

↳ measures

3. JOIN OPERATIONS

• when multiple cubes are present.

Sales	Price
UnitsSold	UnitPrice

* if turnover is not present, then we can calculate it

with the help of unit price

* for revenue, we need both so unit price * unitsold.

• they can be joined if they have same granularity

$$C_1 \bowtie C_2 = C(GM_1, VM_2)$$

• when granularities are different, aggregation should be performed.

Sales & Inventory → Sales (Day, Article, Store, client) to Sales (Month, Article, Store, client)

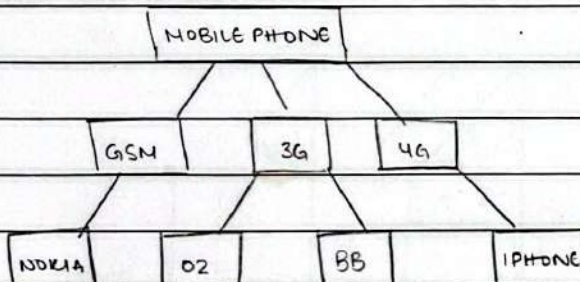
Inventory has months so smaller granularity i.e. days will be converted to months in Sales.

4. AGGREGATION

• SUM, AVG, Count, Min, Max

• SUM (P.Prod-Group, G-city, T-Months) (Sales)

CLASSIFICATION HIERARCHY



• before 3G, O2 was GSM capable,

but the data/connection is now lost

after 2008 since 3G has become more

affordable.

• important to trace the evolution of the data, can explain which data was available at which moment in time.

• using timestamps, annotate data by adding start and end data

Dated:

- tree can be stored as dimension metadata.
- storage form is a validity matrix.
- rows are parent nodes, columns are child nodes.

SCHEMA MODIFICATION

- deleting a dimension can lead to data loss, inconsistencies.
 - proper schema modification is complex but brings flexibility for the end user
- ↳ As Is VS As Was → according to validity matrix and check be

1. SCHEMA EVOLUTION

- modifications can be performed but data may be lost
- involves data adaptation to the new schema also called instance adaptation.

Prod-Cat	Prod-Fam	Prod-Grp	Article	SALES
				TURNOVER.

- advantage: faster to execute queries in DW with many schema modifications.
- disadvantage: limits user to query based on the past schemas.

2. SCHEMA VERSIONING

- no data is lost
- all the data corresponding to all schemas are always available.
- advantage: allows higher flexibility for past even if it is not present in new schema.
- disadvantage: longer query runtime

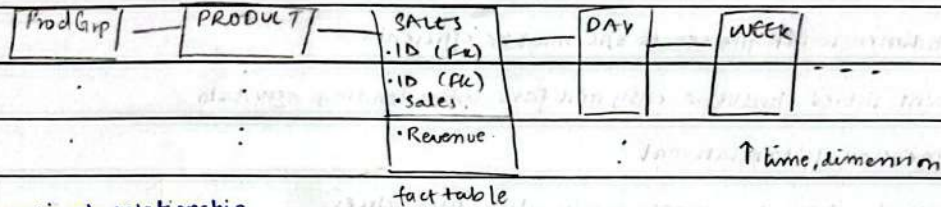
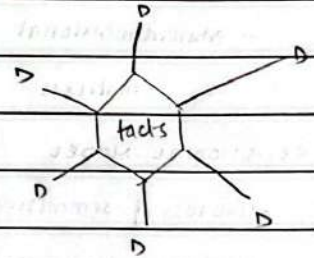
if a business analyst is not interested in Prod-Family, I still have to calculate and aggregate Prod-Family if schema looks like this for the data warehouse, hence unnecessary operations. so just deleting is not an option as data might be lost.

Dated:

SNOWFLAKE SCHEMA

- a table for each classification level
- includes ID of classification level and other attributes
- they are connected by 1:n relationship e.g. n Days to 1 Month.
- measures of cube are maintained in a fact table.
- foreign key ID's for the smallest classification level.

- facts/measures in the centre
- dimensions spread out in each direction along with granularity



- normalized relationship.
- more dimensions, more joins, higher the granularity.

advantages

- ↳ size of dimension table will be reduced and queries run faster.
- ↳ if dimension is very sparse (null data)
- ↳ dimension has long list of attributes which may be queried.

disadvantage

- ↳ fact tables responsible for 90% of the storage requirements. (due to FK)
- ↳ normalization of dimension tables can reduce the performance of DW because it leads to large number of tables.

Dated:

STAR SCHEMA

- use denormalized schema for all dimensions.

- lesser joins are used.

- advantages

- ↳ improves query performance for often used data.

- ↳ less tables and simple structure.

- ↳ efficient query processing with regard to dimension

- disadvantage

- ↳ dimensions are deep, high overhead of redundant data.

SNOWFLAKE VS STAR

- Snowflake

- ↳ structure of classifications are expressed in table schemas.

- ↳ fact table and dimension table are normalized, not for if query needs high level of granularity.

- Star

- ↳ entire classification is expressed in one table.

- ↳ dimension table, the normalization is broken.

- ↳ star schema is a bad idea for big dimension.

SNOWFLAKE TO STAR SCHEMA

- Heuristics based-Decision

- ↳ when typical queries relate to lower level of detail

- ↳ when volume of data in dimension table is relatively low compared to fact table.

- ↳ when modification on the classification are rare compared to insertion of fact data.

* UB Trees - Universal B Trees.

* combined B-Tree and Z-curve.

* vertical partitioning.

Dated: _____

STARFLAKE SCHEMA

- Frequency → if dimensions change often, normalization leads to better result. *small table*
- Amount → bigger the dimension tables, the more space normalization saves. *small table*
- Number of classification level → more classification level introduce more redundancy in star schema. *small table*
- Materialization of aggregates → if aggregates are materialized, dimension can bring better response time.

SCHEMAS

- galaxies → more fact tables, same dimension for all fact tables
- fact constellations → pre-calculated aggregates.
- factless fact table → do not have non-key data

RELATIONAL MODEL DISADVANTAGES.

- representation of multidimensional data can be implemented with a finite set of transformation steps.
- multidimensional queries have to be translated to relational representation.
- to a direct interaction with relational data is not fit for the end user.

2. MULTIDIMENSIONAL MODEL

- multidimensional data stored as array.
- each cube has set of dimension and measures.

→ Linearization

		1	2	3	4
D ₂	5	6	7	8	
	9	10	11	12	

$$D_1 [3], D_2 [2] = 1 \times 4 + 3 = 7$$

D₁

$$\text{index}(z) = x_1 + (x_2 - 1) \times |D_1| + (x_3 - 1) \times |D_1| \times |D_2| + (x_n - 1) \times |D_1| \times \dots \times |D_{n-1}|$$

index of D₁

→ Generalization

Dated:

PROBLEMS IN ARRAY STORAGE

- influence of order of dimensions in the cube definition;
- after linearization, information is spread among more data blocks.
- for a query of all products sold in January, 1 single block is to be read.
- for a query of all sold pants, involves reading 4 blocks that is 16 cells.
- solution → use caching techniques.

↳ performed by the OS
↳ MDBMS has to manage its cache such that OS doesn't perform any damaging swaps.

• storage of dense cube

↳ if cubes are dense, array storage is more efficient

• solution

↳ store dense cube not linear, but on 2 levels

↳ first level are the indexes, second level is data cells stored in blocks.

↳ optimization procedures like indexes, physical partitioning, and compression.

• storage of sparse cube

↳ all the cells of cube, even nulls have to be stored.

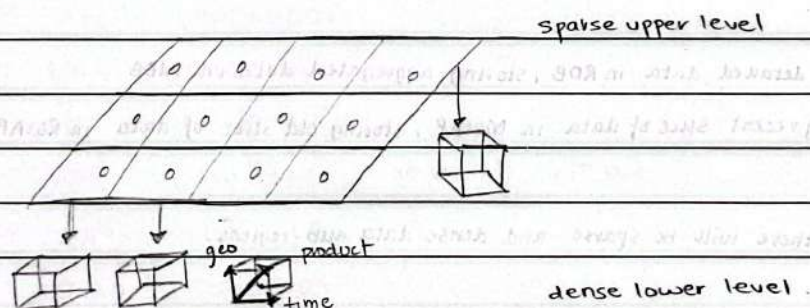
↳ it leads to data being stored in many physical blocks and null values will also have

to be traversed, query speed will be affected.

• solution

↳ don't store empty blocks or pages but adapt index structure.

↳ 2 level : upper layer all possible combinations, lower layer dense dimension.



Dated:

DOLAP

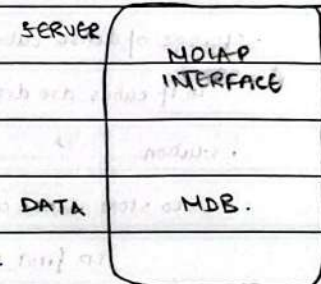
- downloads a small hypercube from a central repository
- computation occurs on the client so it is cheap
- not able to perform analysis because you're limited to what you have on your desktop

MOLAP

- presentation layer provides multidimensional view.
- it is transparent to all the computation done by MOLAP interface and MDB.
- computation occurs at loading step and not at query.



- fast data retrieval, no joins
- optimal for slicing and dicing.
- complex calculations.



- all calculations are pre-generated when cube is created.
- disadvantage → can handle limited amount of data due to large calculation.

→ only summary level information will be included in cube.

→ an input file of 200MB can go to 5 GB with calculation.

ROLAP

- star or snowflake schema to store data.
- ROLAP server gets data what the user inputs, generates SQL queries from OLAP and it requests to query RDBMS.
- indexes : bitmap, R-Trees.
- can handle large amount of data.

CLIENT

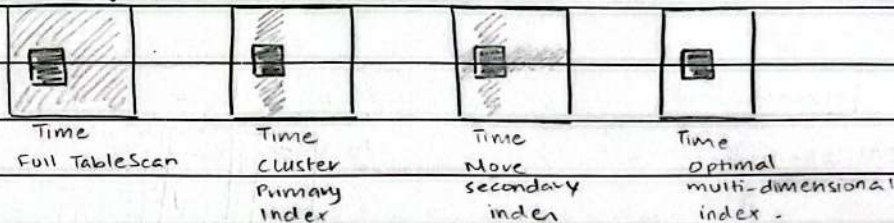
HOLAP

- vertical partitioning → storing detailed data in RDB, storing aggregated data in MDB.
- horizontal partitioning → storing recent slice of data in MOLAP, storing old slice of data in ROLAP
- ↳ Time Dimension
- divide in MOLAP and ROLAP, there will be sparse and dense data sub-regions.

Dated:

INDEXES

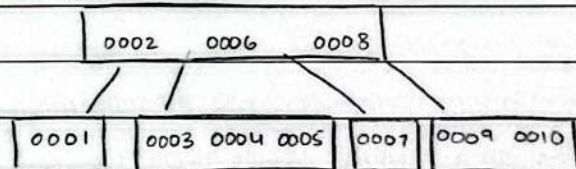
- a 100GB table at 100 MB/s read speed requires 17 minutes full scan which is not best.
- for 30 location, 1000 products, 24 months the selectivity is 1.4KB of data.
- reduce the size of read pages to minimum with indexes.



TREE BASED INDEXES.

1. B-Tree

- basic structure of a node, each node has a key value means the index of the record.
- data structures for storing sorted data for insertion and deletion.



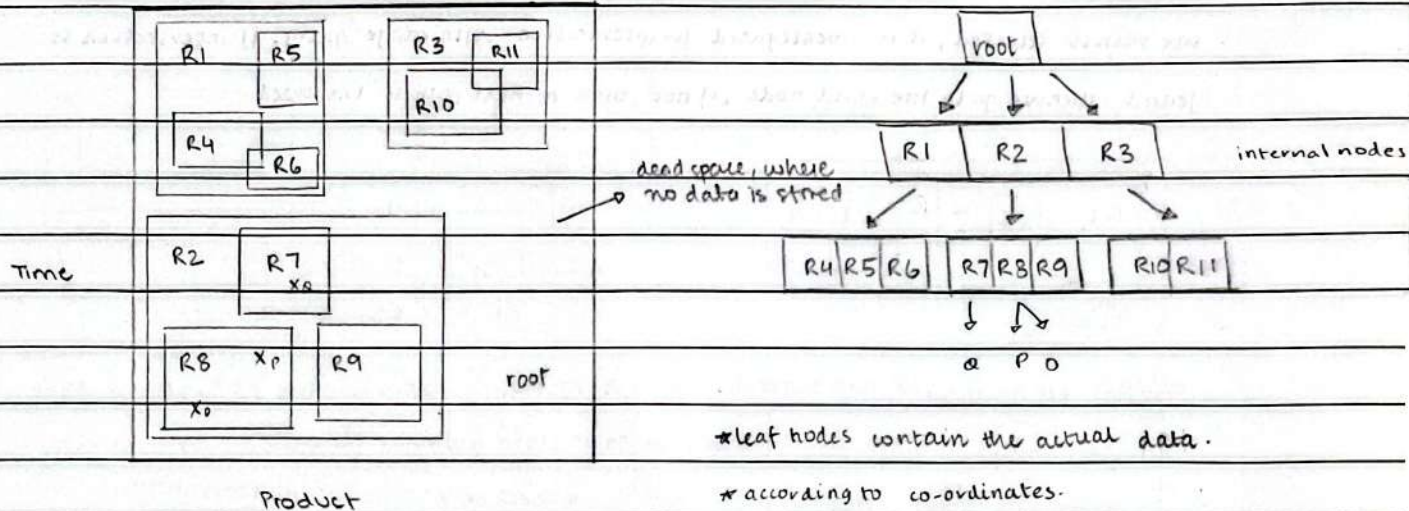
- value smaller than node is on left and greater than node is on right.
- allow exact search with logarithmic costs [self balancing tree]
- DBMS reads whole block in memory even if it needs only one data value.
- search in DW → data is multidimensional, B-trees support one-dimensional search. problem to describe set of points through clusters for actual search and it results in a hierarchical structure.

2. R-Tree

- can go upto dimensions=10 which is enough for small datawarehouse
- dynamic index structure (insert, update, delete)
- data pages are leaf nodes and point to secondary storage where data is located.
- directory pages are internal nodes helpful for navigation purposes.
- Multidimensional data are structured with help of Minimum Bounding Rectangles

Dated:

R-Tree Example.



- local grouping for clustering is not effective for uniformly distributed data.
- allow overlapping clusters but leads to inefficiency while searching.
- height balanced so great for searching as all children have same number of successors.
- objects stored in the leaves, query can be answered without searching all levels.
- MBR used as geometry
- root has at least two children
- internal node has minimum (m) and maximum (M) children.
- M and $m \leq M/2$ are pre-defined

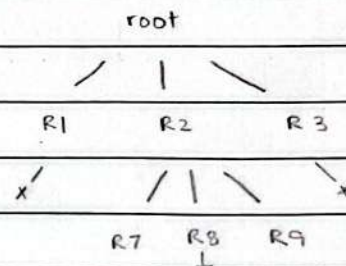
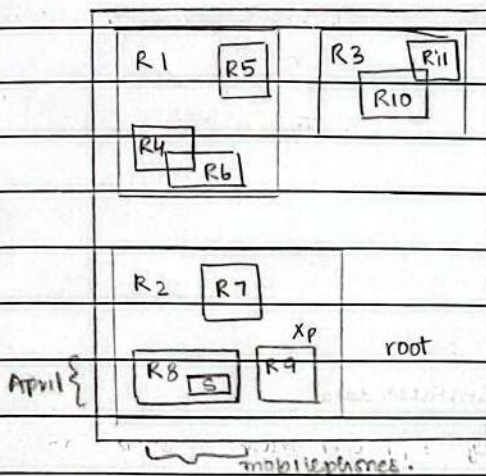
OPERATIONS OF R-TREE

1. Search
2. Insert
3. Update
4. Delete
5. Splitting

Dated:

1. SEARCH IN R-TREE

- starts from root, and searches recursively to the leaves
- one path is selected, it is investigated for intersection with range query. If intersection is found, then we go to the child node, if not then a next path is traversed.

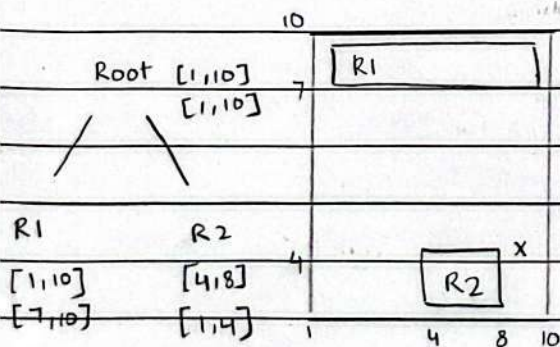


- * check only 7 nodes instead of 12
- * checked R1, doesn't have S, so no need to check children, same with R3.
- * co-ordinates match with R2
- * R3 needs to be checked, after R2 because of overlapping possibility.

- no guarantee for good performance.
- due to overlapping all paths must be traversed.

2. INSERT IN R-TREE

- best leaf node is chosen
- don't include dead space
- object inserted if there is enough room

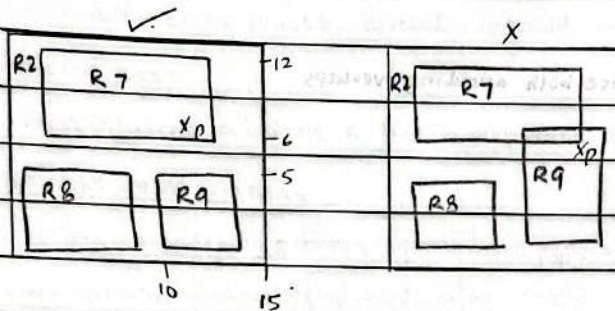


$x \rightarrow [9, 4] \rightarrow 9$ is included in R1, but is very far, hence dead space increases

but for R2, less size is required to increase and add x in the bounding box
need to reduce dead space

Dated:

- if there is no move place left in the node, it is considered overflow and nodes are split then.
- to reduce overlap and have minimum dead space.

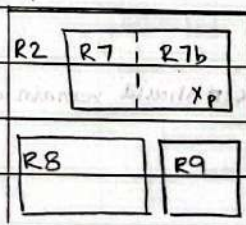


inserting P in either R7 or R9.

$R7 \rightarrow [3, 12] \times$ $R9 \rightarrow [10, 15] \times$
 $[6, 12]$ $[1, 5] \checkmark$
 $P \rightarrow [13, 7]$

- avoid overlaps, R7 given priority even though it takes more space than R9, but R9 has overlaps
- when an object is inserted, expansion needs to be minimum

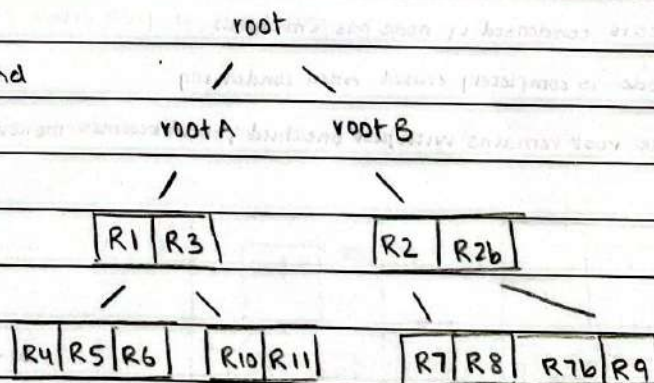
overflow because $M=3$



* suppose R7 has 3 children (S, T, U) already, then a 4th that is x_p cannot come because it violates maximum child condition.

so we split R7 into R7 and R7b but now R2 has 4 children which violates $M=3$ condition so split R2 but now root has 4 children, again violation of condition, so split root to rootA and rootB.

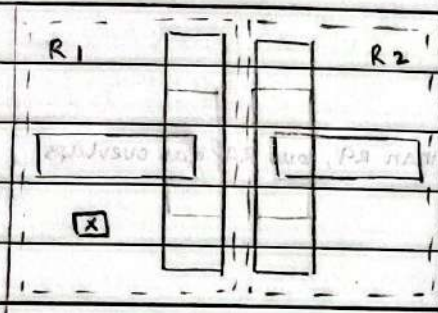
- a horizontal line above R7
- a vertical line between R7, R8 and R7b, R9



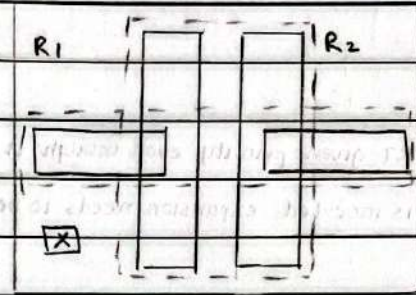
Dated:

3 - SPLIT IN R-TREE

- If an object is inserted in a full node (capacity reached), then $M+1$ objects will be divided among two new nodes
- While splitting, it should ravelly need to traverse both avoiding overlaps.
- use of small minimum bounding rectangle.



bad split

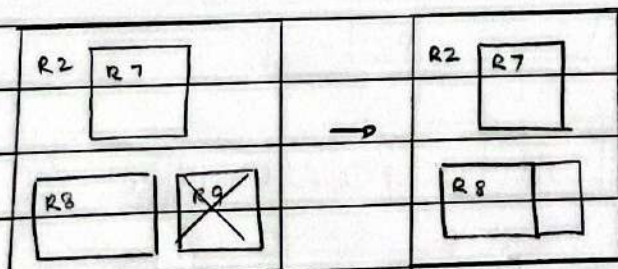


better split

- less dead space is given priority to when splitting.
- overflow problem $\rightarrow$ all objects of old MBR can be divided in different ways, space taken should remain as small as possible.

4 - DELETE IN R-TREE

- search leafnode with the object to delete
- delete object
- the tree is condensed if node has $< m$ objects.
- the node is completely erased when condensing
- if the root remains with just one child, child becomes the new root



- * object from R9 is deleted, so remove R9 as well.
- * then reduce R2 as well

Dated:

S - UPDATE IN R-TREE

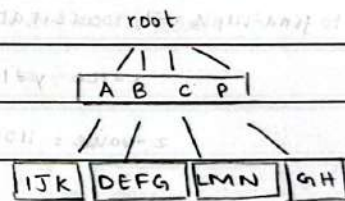
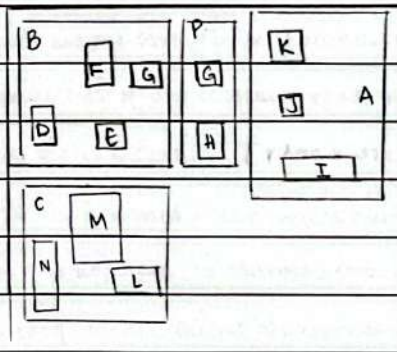
- surrounding rectangle can change
- index entry must be deleted, updated and re-inserted

BLOCK ACCESS COST

- most efficient search in R-tree is performed when overlap and dead space are minimal.

IMPROVED VERSION OF R-TREES

- R-trees allow overlapping between neighbouring MBR's
- R⁺ trees prohibit overlapping of neighbouring MBR's
- ↳ leads to identical leafs occurring more than once.



- more storage space is taken

advantages :

- improves search performance, saves 50% of access time.

disadvantage :

- low occupancy of nodes resulting through many splits.
- they often degenerate

Dated:

UB-TREES

- combination of B⁺ Tree and Z-curve
- z-curve is used to map multidimensional points to one-dimensional values.
- z-values used as keys in B⁺ Tree
- Z-regions

↳ efficient processing of multidimensional range queries.

↳ space covered by an interval on z-curve.

↳ defined by region address of [a:b]

↳ each region maps onto one block of secondary storage

↳ address representation - convert it into binary

	1	2	5	6	17	18
Interval	3	4	7	8		
	9	10	13	14		
	11	12	15	16	27	
	33	34	37	38		

for example to find tuple = 51, locate it at x=4, y=5

$$x = 100 \quad y = 101$$

$$z\text{-value} = 110010 \quad [\text{alternate } x \text{ and } y]$$

↳ why z-values

- reduce dimensionality of data to 1D.
- z-values used as keys in B⁺ trees.
- important for range queries.

↳ range queries

- query can be specified by 2 co-ordinates.

$q_a \rightarrow$ upper left corner

$q_b \rightarrow$ lower right corner

• algorithm

- starts with q_a and calculates z-region

1. z-region of q_a is [0:18]

- corresponding page is loaded and filtered with query

1. tuples 15 and 16.

- next-region inside query rectangle on z-curve is calculated.

1. next jump is on 27

- repeat steps 2 and 3 until end-address of last filtered region is bigger than q_b .

Dated: _____

BITMAP INDEXES

• helpful in datawarehouse

	M/R	Salum	150	bitmap →	Value	Vector
	2	Real	65		P&C	001000
	3	P&C	160		Real	010101
	4	Real	45		Salum	100010
	5	Salum	350			
	6	Real	80		REWE	0000001

AFTER

• costly for OLTP

• record numbers are not changed so if we delete 5, the 6th entry won't become 5

• it would require lots of unnecessary reads if we shift it to 5

• corresponding bit is set to 0

• inserted record is assigned next record number that is

7	REWE	23
---	------	----

• a bit of 0 or 1 is appended to each bit vector.

• when updating change the bit corresponding to the old value to 0 and new value to 1

• for select, basic operations AND, OR are used.

example: select sum spent in salum and P&C

• bitmap indexes should be used when selectivity is high

• advantages

• efficient and easy to implement.

• disadvantage

• each new value of an attribute, a new bitmap vector is introduced means for birthday 365 days / 8 = 46 bytes.

• not fit for range queries where multiple bitmap vectors have to be read. (range encoded)

• multicomponent reduces storage.

Salum	OR	P&C	Result
1	0	1	1
0	0	0	0
0	1	0	1
0	0	0	0
1	0	0	1
0	0	0	0

1 record

2 record

3 record

Dated:

PARTITIONING

- breaking data into several physical unit that can be handled separately.
- granularity and partitioning are key to efficient implementation of a warehouse
- why?
 - managing data is easier on small relevant data.
 - indexing is cheap
 - doesn't need to scan whole table, sequential scan
 - easy recovery and monitoring on less items.
- Datawarehouse, partitioning is done to
 - query performance → less data to scan.
 - data availability → geographically replicated so if one area's system is disrupted, backup is there and can be restored.
 - database administration → adding new columns, archiving data.
- approaches
 1. data partitioning
 - date, line of business, geographical, organizational unit, combination of all
 2. hardware partitioning
 - makes data available to different servers, and that they are capable of running queries.
- which level should data partitioning occur?
 1. application → some processes are important than others so makes sense.
 2. DBMS → database takes responsibility.
 - but not in case of tax laws where it keeps evolving and DW knows nothing.

Dated:

Data Partitioning involves;

1. horizontal partitioning → splitting rows e.g. tax laws, each row represents unique law
2. vertical partitioning → splitting columns. e.g. accounts department doesn't require addresses of customer and is interested in purchase only.

HORIZONTAL PARTITIONING

- set of tuples of a table is split among disjoint table parts.
- no tuples belong in two partitions.
- range partitioning (before, after), list partitioning (top 10), hash partitioning (

1. range

- selects a partition by determining if key is inside the range.
- restriction on master relation.

2. list

- assigned a list of values.
- if partitioning key has one of these values, partition is chosen
- simple restriction on master relation (involves one attribute)

P_1 : CITY IN ('Hamburg')

P_2 : CITY IN (DEFAULT) → tuples not in P_1

3. hash

- for four partition → a value between 0 and 3 can be returned.
- for each tuple in master table, a hash function is calculated.
- useful for sharing load between devices.

• Data Warehouse partitions data by

1. Time Dimension: monthly partitions or by latest or old
2. Other than time: queries run on grouping of data, usually done through list.
3. table size: usually done with hash function

Dated:

VERTICAL PARTITIONING

- create tables with fewer columns
- different physical storage might be used for infrequent used tables.
- also called row splitting.
- creates one to one relationship between partitions and requires a primary key.
- every tuple evenly distributed to every partition.
- Data Warehouse partition data by:
 - moving rarely used data to other, slow devices or another table.
 - creating a view across two newly created tables as joins are very expensive.
 - very large dimension table
 - most of the attributes rarely used e.g. zipcode.
 - maintaining link between fact table and customer so denormalization occurs and move from star to snowflake schema. → high performance cost.
- Solution: MiniDimension

→ many dimension attributes are used frequently so breaking customer into demographic factors and customers itself, one to one relationship.

→ Indexing can be done

CUSTOMER		DEMOGRAPHY	
LASTNAME		AGE	
FIRSTNAME		INCOME	
ADDRESS		AREA	
DEMOG ID		DEMOG ID	

* direct connection with fact table.

* all variables in these mini-dimensions must be presented as distinct classes.

* a foreign key in the fact table and broken dimension table as well.

* should be small as the name suggests.

FACT	
CUSTOMER ID	
DEMOG ID	
PROFIT	
QTY	

Dated:

• advantages

- records are grouped together.
- each partition can be optimized
- security and recovery.
- store partitions on different disks / slower or faster.
- access all partitions at the same time.

• disadvantages

- joins are used, slow retrieval and complex.
- horizontal: union between different resultsets.
- vertical: build tuples.

• partitioning management

- should be transparent outside DBMS.
- only done through logical level and no influence on others.
- database should cover the partitioning, considers data consistency.
- not all DBMS support this.

PARTITION BY RANGE (TIME ID)

PARTITION before 2008

VALUES less than (

PARTITION 2008

VALUES less than ()

ADD PARTITION after 2008

VALUES less than (maxval)

- In data cleaning phase, records can be updated, but partition is already applied so error is displayed.

- Oracle allows data migration meaning;

ALTER TABLE sales ENABLE ROW MOVEMENT → by default it is disabled.

- this deletes the record from previous partition and inserts into other with a new rowid.

→ different harddisk so cannot be read from same ID if different hash index are applied.